

Playing with the Rubik's Cube and its God's number in Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Theiry@inria.fr

Abstract. God's number, the largest number of face turns needed to solve a Rubik's cube, is twenty. It was settled in 2010 by a computation of thirty-five processor years. Even though this kind of computation cannot easily be replicated in a proof assistant like Rocq, three smaller results about the Rubik's cube are proved here. First, we prove that twenty is a lower bound: one position, the superflip, cannot be solved in nineteen. A half turn can also count as two moves. The number is then twenty-six. We prove that twenty-six is a lower bound for solving the four-spot with the superflip on it. This is our second result. Finally, the published computation splits the cube into the 2 217 093 120 cosets of a subgroup, one search to a coset. A coset contains 19 508 428 800 positions. We formalise the correctness of a coset search, and we apply it to one specific coset. It follows that every position of the superflip's coset is solved in twenty moves or less. This is our last result.

Keywords. Rubik's cube, God's number, formal proof, Rocq, group theory.

1 The problem

A Rubik's cube is built from twenty-six small cubes: **eight corner pieces** with three stickers each, **twelve edge pieces** with two, and **six centre pieces** with one. The centres are attached to the core. They spin in place but never travel, so they fix the frame. The white face is wherever the white centre is. A face turn moves four corners and four edges, and nothing else.



Figure 1: The three kinds of piece. Every face shows four corner stickers, four edge stickers and one centre.

Not every arrangement of the pieces can be reached by turning faces. The number of arrangements that can be reached is

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11} / 2 = 43\,252\,003\,274\,489\,856\,000 \approx 4.3 \cdot 10^{19},$$

which reads: the eight corners in any order ($8!$), each twisted one of three ways except that the last is forced by the other seven (3^7); the twelve edges in any order ($12!$), each flipped or not with the last again forced (2^{11}); and a final halving, because corners and edges cannot be rearranged independently of each other.

Turning one face is a *move*, and a half turn counts as one move just like a quarter turn. Every position can be solved. The question is how many moves the worst position needs. That number is called **God's number**.

Counting a half turn as one move is a choice. A half turn can also count as two moves, and that gives a second number for the same cube. We prove a lower bound for each.

In 2010 Rokicki, Kociemba, Davidson and Dethridge showed that it is **20** [1]. Their result has two halves. One half is that twenty moves always suffice. It is the huge one, and the last section of this note says how it was obtained. The other half is that twenty moves are sometimes needed. For that it is enough to take one position and show it cannot be solved in 19.

We take one position: the **superflip**, drawn in Figure 2 beside a solved cube. Every corner sticker is where it belongs. Every edge is in its own place but turned over, so it shows the colour of the face beside it. Look at the cube from any angle, or in a mirror, and the pattern is the same. The superflip is one of the rare positions that all 48 ways of looking at a cube leave unchanged, and that matters later. A 20-move solution for it is known, so ruling out a 19-move solution puts the superflip at distance exactly 20. That is what we prove.

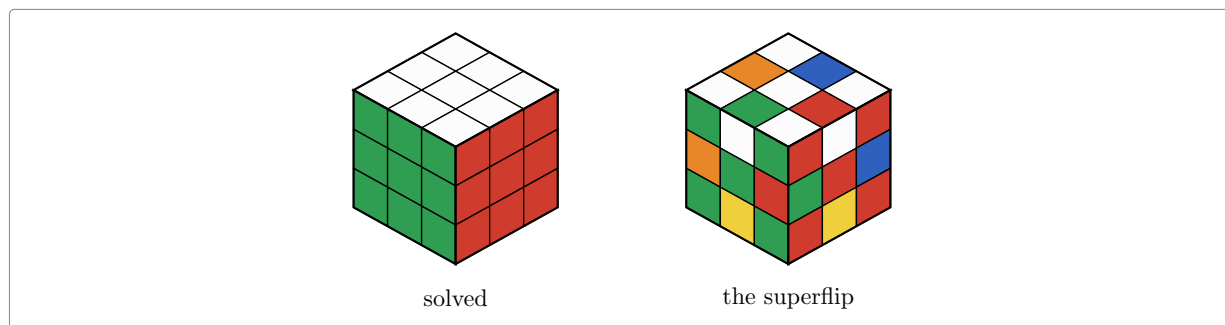


Figure 2: A solved cube, and the superflip.

That is still a big computation. There are 18 moves at each step, so 19 moves means about 18^{19} words.

Three key facts about a position are used in what follows.

- Each corner has exactly one sticker of the top colour or the bottom colour. That sticker can be in three places on the corner. It can be on the top or bottom face, which we write 0, or on one of the corner's two sides, which we write 1 and 2. We call this the corner's **twist**.
- Each edge has a right way round, which we write 0. Put back the other way round, it shows its two colours the wrong way about, and that we write 1. That is what we call the edge's **flip**.
- Four of the twelve edges belong in the middle layer, between the top and the bottom face. We call that layer the **slice**. There are $\binom{12}{4} = 495$ ways of choosing which four of the twelve edge slots those four edges are in, and we number them 0 to 494, with 0 for the four slots of the slice itself.

The three are easy to read on the superflip. Every corner is home and the right way up, so every top or bottom sticker is on the top or bottom face and its eight twists are all zero. Every edge is turned over, so its twelve flips are all one. Every edge is also in its own slot, so the four middle edges are back in the four slots of the slice, which is 0.

2 The cube as permutations

The cube is easier to reason about if we stop treating it as a solid object. Only the coloured stickers matter. There are six faces of nine stickers, and the six centre stickers never move relative to each other. So a move is a rearrangement of the **48 remaining stickers**. We number them 0 to 47, as in Figure 3 and Figure 4: eight to a face, taken left to right and top to bottom with the centre skipped, so up gets 0–7, left 8–15, front 16–23, right 24–31, back 32–39 and down 40–47. The sources use these numbers, so a move can be checked against a picture.



Figure 3: Three of the six faces, and the turn of the top one.



Figure 4: The cube unfolded, with all forty-eight places numbered.

With the places numbered, a move is written down by saying where the sticker in each place goes. Turning the top face clockwise carries the sticker in corner 0 to corner 2, the one in 2 to 7, the one in 7 to 5 and the one in 5 back to 0. That is a four step cycle, written (0 2 7 5). The four edge stickers of that face do the same, (1 4 6 3). The turn does not only move the top face. It also carries the top row of each side face round to the next one: front to left, left to back, back to right, right to front. That is three more cycles, (8 32 24 16) and its two companions. Figure 5 shows all of it, each square saying which sticker sits there afterwards. The top row of the left face holds 16, 17, 18, the stickers that came round from the front. The other forty stickers stay where they are, and the six centres never move.



Figure 5: Everything a clockwise turn of the top face moves.

Six clockwise quarter turns generate everything: up, right, front, down, left and back. Each of them can also be done twice or backwards, which gives the **eighteen moves**

$$\begin{array}{cccccc} U & R & F & D & L & B \\ U^2 & R^2 & F^2 & D^2 & L^2 & B^2 \\ U^{-1} & R^{-1} & F^{-1} & D^{-1} & L^{-1} & B^{-1} \end{array}$$

A position is a product of moves, for instance $RUR^{-1}U^{-1}$, a word of length 4. The set of all positions is a group G : the **cube group**. Solving a position in d moves means writing it as a word of d moves. So “solvable in at most d moves” says that the position lies in the **ball of radius d** around the solved cube. God’s number is the largest distance that occurs.

2.1 The cube in Rocq

The development is written in the Rocq prover [2] and built on **mathcomp** [3], a large library of formalised mathematics that already has permutations, groups and products. The cube file, `Rubik333.v`, is a transcription of the paragraphs above, and it is short:

```
Definition facelet := 'I_48.
```

```
Definition Umove : {perm facelet} :=
  cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@; 6@; 3@] *
  cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] *
  cyc [:: 10@; 34@; 26@; 18@].
(* ... and five more, one per face ... *)
```

```
Definition faces : seq {perm facelet} :=
  [:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].
```

```
Definition moves : seq {perm facelet} :=
  flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].
```

```
Definition G : {group {perm facelet}} := <<Sset>>.
```

Here is what each line says:

- `'I_48` is the type of the whole numbers **below** 48, so the places are numbered **0 to 47** and not 1 to 48, everywhere in the sources and in the pictures of this note.
- `{perm facelet}` is the type of **permutations** of those places: a way of sending each place to a place, no two of them landing on the same one. That is exactly what a position is.
- `cyc [:: 0@; 2@; 7@; 5@]` is the **cycle** that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other forty-four places where they are. The @ is local notation turning a plain number into a place.

- $*$ composes two permutations, so **Umove** is the five cycles of Figure 5 done together, and g^{+2} and g^{-1} are the same turn done twice and undone. Its order is the opposite of the usual one, and it catches the reader who knows how functions compose. Mathcomp applies permutations on the right, so $(g * m) f$ is $m (g f)$: a product reads left to right, like a sequence of moves played one after the other.
- **seq** is a list, and **faces** is the list of the six clockwise quarter turns. **moves** runs through it and keeps three moves per face, which is the eighteen.
- **<<Sset>>** is the group generated by a set: everything reachable by composing moves, which is the cube group.

The superflip is written down the same way in **Diameter.v**, as the twelve swaps that exchange the two stickers of each edge:

```
Definition Spcyc : seq (seq facelet) :=
  [:: [:: 1@; 33@]; [:: 3@; 9@]; [:: 4@; 25@];
    (* ... nine more, one per edge ... *) ].
```

```
Definition superflip : {perm facelet} := \prod_(l <- Spcyc) cyc l.
```

That makes it a permutation of the stickers, but on its own it says nothing about the cube. A permutation is a legal position only if the faces can be turned to reach it, that is, only if it lies in G . The definition above does not give that. It has to be proved.

The proof is one equality. On the left, the superflip as just defined. On the right, a word of twenty moves:

$$U R^2 F B R B^2 R U^2 L B^2 R U^{-1} D^{-1} R^2 F R^{-1} L B^2 U^2 F^2$$

Both sides are permutations of the 48 stickers. Each is written out as the list of the 48 places it sends each place to, so the equality is one comparison of two lists. Every letter on the right is one of the eighteen moves, so the superflip lies in G . That same word gives the upper bound of 20 for this one position.

Nothing here is assumed. There is no axiom saying what a cube is. A reader who wants to check the model has only to compare the six lists of cycles against Figure 4. After these two files, stickers are never mentioned again.

3 Searching for the lower bound

We want to know that the superflip cannot be solved in nineteen moves. The naive way is to try every word of at most nineteen moves, starting from the superflip: if the solved cube never turns up, the lower bound is twenty. A tree of depth 19 with a branching of 18 is 18^{19} words, far too many to make the search practical. This naive method has to be refined.

3.1 The pruning estimate

A first refinement is to have a criterion that lets us cut in advance the branches we are sure cannot succeed. For that we use at each position a cheap lower bound h on the number of moves still needed: if h is 20 while only 18 moves remain, the branch cannot reach the solved cube in time and is cut with everything below it, as Figure 6 shows.



Figure 6: The search, and its scissors.

Such a lower bound is called an **admissible** estimate, and a depth-first search that deepens step by step and prunes on one is Korf's IDA* [4].

3.2 How to get a cheap estimate

The estimate has to come from somewhere, and the idea is to forget most of the cube. Keep only part of the information, say how the corners are twisted and where the four middle-layer edges sit, and call what is left a **summary**. Many positions can share the same summary. Moves act on summaries as well as on cubes, and summaries are few enough to compute the exact distance of each one to the solved summary and keep them all in a table. This gives h : take a position, compute its summary, and look its distance up in the table. This idea of summary comes from Culberson and Schaeffer, who call such a table a *pattern database* [5], and Korf solved the cube optimally with three of them [6].

The summary we have used is Kociemba's, from his two-phase solver [7]. Figure 7 shows the three things it records. A corner has one sticker belonging to the up or down face, and that sticker sits in one of three places, which is 0, 1 or 2. An edge is either the right way round or turned over, which is 0 or 1. The four edges of the middle layer occupy four of the twelve edge slots. The figure shades them on the two visible faces, where three of the four can be seen. Nothing else about the position is recorded.



Figure 7: The three things a summary records.

The summary is the product of the three values:

summary	values	
how the eight corners are twisted	2 187	$= 3^7$
how the twelve edges are flipped	2 048	$= 2^{11}$
where the four middle-layer edges sit	495	4 places among 12
the summary, all three together	2 217 093 120	

Every summary stands for exactly **19 508 428 800 real positions**. The table records, for each summary, its distance from the solved summary. Four bits hold one entry and the whole table is **1.18 GB**. The cut is quite effective. A search at depth 14, for instance, visits 470 786 nodes, out of the $1.07 \cdot 10^{15}$ the same tree holds without it: one node in two billion. In the following, we call this summary the **phase 1 summary**, after the first phase of Kociemba's solver [7], and **phase 1 table** the table of its distances.

4 The search in Rocq

4.1 The generic search

The search at depth d is implemented in Rocq by a generic search, `Search.v`, about a hundred lines that never mention the cube. It is given a group, a set of moves and an estimate h , with only two assumptions attached to h :

Hypothesis `h1` : $h\ 1 = 0$.

Hypothesis `hstep` : $\text{forall } g\ m, m \in S \rightarrow h\ g \leq (h\ (g * m)).+1$.

```
Fixpoint search (d : nat) (g : gT) : bool :=
  (h g <= d) &&
  ((g == 1) ||
   (if d is d'.+1 then has (fun m => search d' (g * m)) Sseq else false)).
```

Corollary `searchN d g` : $\text{search } d\ g = \text{false} \rightarrow g \notin \text{ball } S\ d$.

Again, here is what each line says:

- `gT` is the group the file works in. It is a variable, so nothing here is about the cube; the cube is what it gets instantiated with later.
- `1` is the unit of that group, which for the cube is the **solved** position. So $h\ 1 = 0$ says the estimate of a solved cube is zero, and $g == 1$ asks whether the search has arrived.
- `Sseq` is the list of moves, the eighteen of them, in the order the search walks over them. `S` is the same thing seen as a set.
- $g * m$ is the position g followed by the move m , in the left to right order met above.
- $h\ g \leq d$ is the cut, and $(h\ (g * m)).+1$ is the estimate after a move plus one, which is the assumption that one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move available, and answers as soon as one of them succeeds.

The last line exactly states what we need about the search: **if the search returns false, the position is not in the ball of radius d .**

4.2 The summary and its table

The estimate h is built in a second generic file, `Coord.v`. It is given the summary of a position, the way a move acts on a summary, and the table of distances. Three hypotheses come with them, one about the summary and two about the table.

Variable `coord` : $\{\text{perm facelet}\} \rightarrow X$.

Variable `act` : $X \rightarrow \{\text{perm facelet}\} \rightarrow X$.

Hypothesis `coordM` : $\text{forall } g\ m, \text{coord } (g * m) = \text{act } (\text{coord } g)\ m$.

Variable `D` : $X \rightarrow \text{nat}$.

Hypothesis `D0` : $D\ (\text{coord } 1) = 0$.

Hypothesis `Dstep` : $\text{forall } x\ m, m \in Sset \rightarrow D\ x \leq (D\ (\text{act } x\ m)).+1$.

X is the type of summaries and `coord` gives the summary of a position. `act` plays a move on a summary directly, without going back to the position it came from, and `coordM` says that the two agree: playing a move and then summarising gives the same answer as `act`.

The `search` of `Search.v` is refined to carry the summary beside the position, updating it with `act` at each move. The position is still carried, since only it tells whether the cube is solved. The refined search also plays fewer moves: turning the same face twice running is never useful, and the other rules of that kind are the subject of the next subsection. Here is its shape, with names simplified and some details left out; the real one is `searchz3` in `Farp1.v`:

```
Fixpoint search (d : nat) (g : gT) (x : summary) (p : move) : bool :=
  (D x <= d) &&
  ((g == 1) ||
   (if d is d'.+1
    then has (fun m => search d' (g * m) (act x m) m) (allowed p)
    else false)).
```

Two things travel down the tree instead of one, and each is used for exactly one job:

- `D x <= d` is the cut. It reads the table at the summary `x`, and never looks at the position.
- `g == 1` asks whether the cube is solved. It reads the position `g`, and never looks at the summary.
- `g * m` moves the position and `act x m` moves the summary, side by side, one move at a time. That step is `coordM` being used, and it is why the summary never has to be recomputed from the position.
- `p` is the move just played, and `allowed p` is the list of moves the rules permit after it. That is where redundant sequences are dropped.

4.3 The table and its two conditions

For the phase 1 summary, `D` is a lookup in the phase 1 table, so `D0` and `Dstep` become two statements about that table:

- the entry of the solved summary is zero;
- every entry is at most one more than the entry reached from it by any of the eighteen moves.

The table is generated by an OCaml program, and it gives the exact distance. The proof does not need that. The two statements above are all it checks, and they are enough: together they make the estimate a lower bound on the moves still needed, which is what lets a branch be cut. A table of zeros would pass both. It would prune nothing and the search would run for ever, but the answer would still be right. So the generator is not trusted. It writes the table out as Rocq source, and the two statements are checked on it afterwards. The entries do not depend on each other. So the second check is cut into slices, one file each, and the slices are checked at the same time. It takes ten minutes. Every timing in this note is measured on the same machine, the **reference machine**: a dual-socket Intel Xeon E5-2667 at 2.9 GHz, twelve cores, twenty-four threads, 62 GB of memory.

5 Optimising

What follows makes the tree smaller or the run cheaper. None of it changes the answer.

5.1 Removing redundant moves

Many words lead to the same position, and the search does not have to try them all. Two ideas say which ones may be left out. The first is repetition: after a U there is no point in trying U , U^2 or U^{-1} , since UU is U^2 and a shorter word is covered at a smaller depth, which leaves fifteen moves instead of eighteen. Opposite faces are the same argument, one step weaker: UD and DU give the same position, so of the two orders we keep one, and play the top, right or front face first. This is what we call the **order convention**. From the third move on it leaves **twelve** moves after a turn of the top, right or front face, and **fifteen** after a turn of the bottom, left or back one. The second idea

is symmetry. The superflip is unchanged by all 48 relabellings of the cube. So we need to explore only the turns of one face for the first move. We choose arbitrarily the top one, and again by symmetry we only have to consider U and U^2 , since U^{-1} is the symmetric of U . The two ideas collide at the second move. After U , repetition removes U , U^2 and U^{-1} , which leaves fifteen. The order convention would remove D , D^2 and D^{-1} as well and leave twelve, but it may not be used here: the first move is already fixed to the top face, and turning the cube upside down takes DU back to UD . So the bottom face stays, and the fifteen second moves after U include UD , UD^2 and UD^{-1} .

The search is then parallelised at depth two. Two first moves times fifteen second moves is thirty **prefixes**, each searched on its own to depth 17, and they are packed one file per second move, `Runp1_03.v` to `Runp1_17.v`. The two whose second move turns the bottom face keep fifteen branches where the others keep twelve, so they run far longer. Each of those two is split into two files, `Runp1_09a.v` and `Runp1_09b.v`, `Runp1_11a.v` and `Runp1_11b.v`, which balances the load and makes **seventeen files** in all. Our own OCaml program dropped the bottom-face moves, so it searched 24 prefixes where it had to search 30. It ran for hours and gave the answer we expected. The error came out only when the cut had to be proved in Rocq, and the proof could not be written. A cut that is too greedy does not make a search fail. It makes it faster, and it makes it agree with you.

5.2 The effective representation

To run the search we need an effective representation of its objects: of a position first of all, and of the permutations that move it. The objects are few: a position, a move, the summary of a position, and the table of distances. A position is a permutation of the 48 stickers, and `Table.v` presents it by its image table, the list of 48 numbers saying where each sticker goes, with `tab_ok` saying which lists are tables; the product of two permutations is then the reading of one list through the other, and a move is one more table. `Tsearch.v` runs the search of `Search.v` on tables. Machine integers, 63 bits wide, and **persistent arrays** of them [8] come next: `Tabi.v` carries those tables as arrays of machine integers, `ti2t` reads such an array back as the list it stands for, `tabi_ok` is `tab_ok` of that list, and each operation has a lemma saying that the bridge may be crossed either way round.

```
Lemma ti2t_comp a b :
  tabi_ok a -> tabi_ok b ->
  ti2t (comp_tabi a b) = comp_tab (ti2t a) (ti2t b).
```

From there on a position is 48 machine integers, a summary is two, and the phase 1 table is an array of arrays, fifteen four-bit entries to a 63-bit machine integer. A Rocq array holds at most 4 194 303 entries. The table needs far more, so it is cut into chunks of two million words. A function on a finite domain is tabulated rather than computed, the action of a move on a summary, the rank of a summary and the symmetry that the fold uses among them, and each of those tables is checked in Rocq like the table of distances. The search itself goes the same way: `Fast.v` holds it twice, `searchz3` on the objects of the last section and `searchz3n` on machine integers and arrays, and `searchz3nE` in `FastP.v` proves that the two answer alike, asking only that the depth fit in a machine integer, that the tables have passed their checks, and that the array be a well-formed position. Seven versions lie between the two ends, each proved equal to the one before, and together they are 11.9 times faster on one piece at depth 14.

The superflip itself goes down that chain. As a permutation it is a product of twelve two-cycles, one for each flipped edge, (1 33), (3 9), (4 25) and so on. `Moves.v` turns those cycles into the image table `sftab`, and the table into the array `sfti` the search starts from. Each step has its lemma:

```
Lemma sftabE : superflip = pt 47 sftab.
Lemma sftiE : superflip = pt 47 (ti2t 47 sfti).
```

`pt 47` is the permutation a table stands for, so both say that what runs is still the superflip. Its summary is read off the same table, the corner twist by `ctwistt` and the flip-and-slice value by `coordt`, and the estimate at the root of the search is one expression:

`Dpli (ctwistt sftab) (coordt sftab)`

The summary is (0,15 732 735): the superflip leaves the corners alone, so the twist is zero, and the second number carries the twelve flipped edges and the four slice slots. The lookup goes through the fold to a four-bit field of one 63-bit integer, and the value there is **ten**. At the root the search therefore knows that at least ten moves are needed, and it has nineteen to spend, so nothing is cut there.

Functions are treated the same way. A function on a finite domain is tabulated once and then read: the action of a move on a summary, the rank of a summary, and the symmetry that the fold uses are all arrays, not computations. The search reads them where the mathematical text applies a function, and each table is checked in Rocq like the table of distances.

The search itself goes the same way. `Fast.v` holds it twice. `searchz3` is the abstract one, written on the objects of the last section. `searchz3n` is what runs, on machine integers and persistent arrays. `FastP.v` ties the two:

```
Lemma searchz3nE T d a p :
  (d <= 63)%N -> fsmoveC -> (p < 7)%N ->
  tabi_ok 47 a -> cubti a -> twP3 a ->
  searchz3n T d (of_nat d) a [::] (init3 a) p
  = searchz3 T d a (init3 a) p.
```

The hypotheses say that the depth fits in a machine integer, that the move table passed its check, and that the array is a well-formed position. Between the two there are seven versions, each proved equal to the one before, and together they are 11.9 times faster on one piece at depth 14. Only the ends are of interest here: whatever the middle versions do for speed, the answer is the abstract search's.

5.3 Two uses of symmetry

The first relabels the position. Rotating the whole cube about a corner axis gives the same position seen differently, and its summary is then another entry of the same table. Each of the three views therefore gives a lower bound on the number of moves left, and so does the largest of the three. That costs three lookups at a position instead of one and buys a sharper cut and a smaller tree. Cube solvers do this as a matter of course, Kociemba's included; what is new here is that the three views are proved legitimate.

The second relabels the table. The summary is built around the up-down axis: the twist records where each corner's up-or-down sticker sits, the slice where the four edges between the top and bottom faces are. Sixteen of the 48 relabellings keep that axis and turn one summary into another, and they sort the 1 013 760 flip-and-slice values into **64 430 families**, a factor of **15.73**. Two values in one family are the same distance from solved, so one entry per family is enough. The estimate is therefore unchanged, and only the table shrinks. In the code the change is one definition. The lookup was

Definition `Dpli (tw x : int) : int := plget (plidx tw x).`

and it became

Definition `Dplri (tw r : int) : int :=`
`plget (plfoldi (frep r) (twsym tw (fsym r))).`

`frep r` is the family's representative, `fsym r` the symmetry that carries the value to it, and `twsym tw (fsym r)` the twist carried through that same symmetry. Three more reads at every lookup, into a table 15.73 times smaller.

Symmetry-reduced tables are standard in cube solvers. What the development adds is a proof that the folded table still passes **D0** and **Dstep**, and that is all it has to prove: conditions demanding true distances would have required a proof that the fold preserves them, a harder statement about the sixteen symmetries and about what sharing an entry between two summaries does. The check is run on the folded table as it was on the flat one, and it is the same check. The fold costs the search 1.61

times at depth 16 and pays everywhere else: a search worker drops from 4.15 GB to **0.85 GB**, so all the pieces run at once instead of in two waves, and checking the table drops from about 5.4 processor hours to **1.35**.

6 The development and its cost

The statement proved at the top of the chain is

Theorem `superflip_plfar_real` : `superflip \notin ball Sset pldepth`.

In words, the superflip is not within `pldepth` moves of the solved cube, where one script sets the depth before the run. It has **no hypotheses left**, and nothing in the chain is admitted: asking Rocq what the proof assumes reports only the primitives of its machine-integer and array interface. At depth 19 it says that the superflip cannot be solved in 19 moves, and two lines in `Diam20.v` turn that into **God's number ≥ 20** , after checking that the searches really were run at 19.

It is forty-six hand-written files, 12 725 lines about the cube and 1 308 more of machine-integer toolbox, beside 156 000 lines of generated tables in the repository and 165 MB of them too big to store. The cube and its symmetries are `Rubik333.v`, `Sym.v` and `Ball.v`, the abstract search `Search.v` and `Coord.v`, the representations `Table.v` and `Tabi.v`, the summary and its table `Coordfs.v` and `Phase1.v`, and the search on the real data `Farpl.v`, `Fast.v`, the seventeen pieces `Runpl_03.v` to `Runpl_17.v` and `Diam20.v`. The checks live in certificate files of their own, each behind its own `Qed`, and `README.md` lists everything with the scripts that run it.

Building the tables costs the same whatever radius is searched afterwards. Measured end to end from a clean tree on the reference machine:

	wall clock	processor time
emitting the tables and compiling them to native code	17 min 21	52 min 13
the coordinate and summary tables	22 min 46	21 min 15
the search and the estimate, over no data at all	11 min 02	30 min 45
the four certificates for the move and distance tables	56 s	2 min 27
the fold: twelve checks and twenty-seven slices	9 min 52	47 min 57
the shape check against the dummy table	18 s	16 s
in total	1 h 02	2 h 35

The second line runs on one core, so more cores do not help. The first and fifth are mostly the OCaml compiler turning a table into native code, and together they take 100 of the 155 processor-minutes.

The search itself visits 146 065 078 152 positions, from 5 575 767 076 in the smallest piece to 10 554 835 820 in the largest, and the tree grows by 12.22 from one level to the next between depths 17 and 19. We measured the run twice, once before the fold and the two reductions and once after. It is the same theorem both times.

radius 19, search depth 17	before	after
pieces	18	17
workers	9	18
memory per worker	4.15 GB	0.85 GB
wall clock	11 h 13	6 h 36
processor time	85 h 11	87 h 36

The fold halves the wall clock and leaves the processor time where it was. What it buys is memory: at 0.85 GB a worker, all seventeen pieces run at once, where 4.15 GB allowed only nine. So the proof costs 87 processor-hours and one night, and that is a measured cost.

7 Counting in quarter turns

Counted in quarter turns there are twelve moves: the six faces one way and the same six back. A half turn is two moves. The answer in that count is **26** (cube20.org). We prove the lower half: one position cannot be solved in 25 quarter turns.

7.1 The position, and 25 down to 24

The superflip is 24 quarter turns from solved, so it is not far enough away. In August 1998 Reid posted a better position to the Cube-Lovers list [9]: the **four-spot** with the superflip composed onto it. That post is the source of this section, and it is transcribed beside this note.

The four-spot exchanges the front and back colours and the left and right colours. The centres cannot move, so each of those four faces keeps its own colour in one square, which is the spot the pattern is named after. The superflip then turns every edge over.

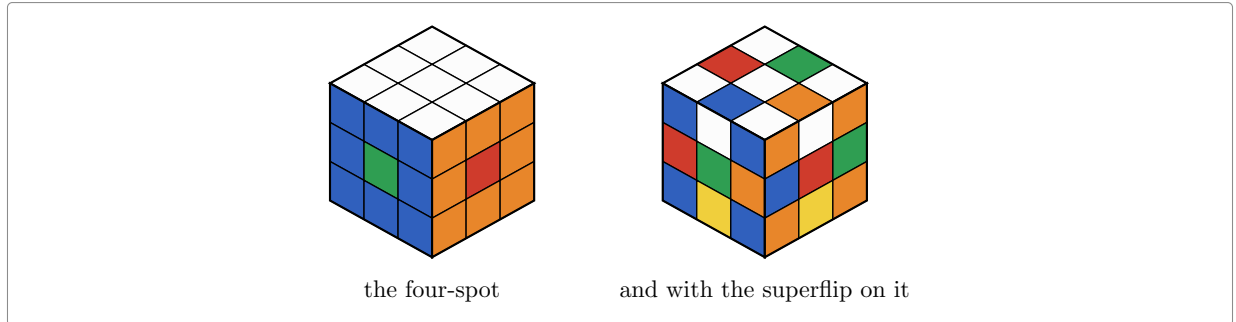


Figure 8: The four-spot, and Reid's position.

Reid's position is 26 quarter turns from solved. His word for it is

$$U^2 D^2 L F^2 U^{-1} D R^2 B U^{-1} D^{-1} R L F^2 R U D^{-1} R^{-1} L U F^{-1} B^{-1}$$

That is 21 face turns, five of them half turns. We check it by multiplying out both sides and comparing two lists of 48 places.

Ruling out 25 is ruling out 24. A quarter turn is five four-cycles of the 48 stickers, so it is odd, and a manoeuvre of odd length gives an odd position. Reid's position is even. So every manoeuvre for it has even length, and the searches stop at 24.

7.2 Reid's six prefixes

The argument has two halves and only the second is a computation.

The first is Reid's Proposition 2: a manoeuvre that cannot be shortened can be rewritten, at the same length, to begin with one of six sequences.

$$\begin{array}{ccccc} U & R & F & D & L & B \\ U^2 & R^2 & F^2 & D^2 & L^2 & B^2 \\ U^{-1} & R^{-1} & F^{-1} & D^{-1} & L^{-1} & B^{-1} \end{array}$$

The rewriting uses three operations that change neither the product nor the length: conjugation by one of the sixteen symmetries that fix the position, inversion, and cyclic shift with the letters that move to the end relabelled. The last is why we chose this position. **HProp2.v** is that argument in Rocq, and **HBridge.v** carries it to the orientation the search uses.

Reid does not state one hypothesis his proof needs: the manoeuvre cannot be shortened. Without it the third turn may cancel the second. The Rocq statement carries the hypothesis at no cost, since a shortest manoeuvre is what we want anyway.

Six searches follow. The first prefix is two turns long and is searched 22 further, the other five are three long and are searched 21 further. Each reaches 24 turns.

7.3 The summary, and its table

The estimate is built as before, by keeping a summary of the cube:

summary	values	
where the four middle-layer edges sit, each of them the right way round or not	190 080	$= 24 \cdot 22 \cdot 20 \cdot 18$
which four corner places hold the four top corners	70	4 places among 8
how the eight corners are twisted	2 187	$= 3^7$
the three together	29 099 347 200	

The 24, 22, 20 and 18 fall by two each time because a place taken by an edge is taken whichever way round that edge is. These summaries are the cosets of Reid's H, which is where the H at the front of the file names comes from.

The table holds the distance from solved of each of the 29 billion summaries. How many lie at each distance agrees with the column Reid published in 1998, and we run that check first. The table is then folded. The sixteen symmetries that keep the up-down axis sort the 190 080 edge values into 12 094 families, a factor of 15.72, and one entry is kept per family. That is 883 MB, measured at 3.86 GB once loaded into the prover.

7.4 The files

Reid's argument	
HCoord.v	the three coordinates of a position, on facelet tables
HRoot.v	Reid's six positions, and the three views of them
HProp2.v	manoeuvres as words, and the three rewritings of them
HReid.v	what Proposition 2 rests on
HBridge.v	from Proposition 2 to the position the run searches
the search	
HSearch.v	the quarter-turn search over Reid's table
HCanon.v	the rule the search plays by loses no manoeuvre
HCut.v	a cut throws no manoeuvre away
HPrefix.v	playing a word is stepping the state
HPok.v	the positions the search meets, and the triples it carries
HRunS.v, HSound.v	a search that fails is a proof that no word exists
the tables, and the assembly	
HChk.v	the move tables against the coordinates
HEdge.v, HCorner.v	a turn acts on the datum, for edges and for corners
HAgree.v	the coordinates agree with the tables, everywhere
HSweepC.v	the three sweeps over the move tables, in six slices
HSweep.v	the sweep over the distance table, cut into twelve jobs
HAdmis.v	what that sweep buys: the estimate is never too big
HGlue.v, HBound.v	what the run has to give, and what it gives
HFinal.v, HAll.v	the bound assembled, and qdiam25

7.5 The theorem, and its cost

Theorem qdiam25 : $\sim \text{diam_le Sq } 25$.

`Sq` is the set of the twelve quarter turns, and `diam_le Sq 25` says every position is within 25 of them. The line says it is not. Reid’s position is the witness, and his word puts it at 26. Rocq reports only the primitives of its machine-integer and array interface.

	wall clock	processor time
building the table, in OCaml	9 min 50	1 h 43
the same table as fifty-nine Rocq files	3 h 13	6 h 50
the three sweeps over the move tables		1 min 20
the sweep over the distance table, twelve jobs	2 h 51	31 h 42
the six searches, seventy-two pieces, twelve workers	4 h 00	45 h 54
the whole chain in Rocq	10 h 32	87 h 29

The last row is not the sum of the others. It is the whole chain measured end to end from a directory where nothing is built. We build the OCaml table on the first row once by hand, and it is not part of that run.

The sweeps and the searches are nearly nine tenths of the cost. Checking the table rather than trusting it costs about two thirds of what the searches cost. The quarter-turn work adds eighteen hand-written Rocq files and 6 008 lines.

8 One coset of the upper half

Everything above is a lower bound, and a lower bound rests on one position and one search that finds nothing: the superflip for the twenty face turns, the four-spot with the superflip on it for the twenty-six quarter turns.

The other half of God’s number, that twenty moves always suffice, is the huge one, and we do not prove it here. `Diameter.v` holds the reduction for it, and that reduction rests on one assumption.

The published proofs do not solve all 43 quintillion positions one at a time. They cut the cube group into the 2 217 093 120 cosets of a subgroup and solve a whole coset at once. One search settles every one of the 19 508 428 800 positions in it. The positions in a coset are not all the same distance from solved. What the search shows is that none of them is more than 20. A symmetry of the cube carries one coset to another, and the image is solved by the same manoeuvres relabelled, so only one coset per symmetry class is searched. Two things are then needed: that the cosets searched cover every class, and that each search really settles its whole coset.

The first is not a computation, and `Canon.v` proves it. Take as representative the least member of each class, in the order the finite type already carries. The covering property holds because a finite set has a least element. It is eighty lines and assumes nothing.

The second is the computation itself, the one Rokicki, Kociemba, Davidson and Dethridge ran: about a billion seconds of processor time, more than thirty processor years, donated by Google, over 55 882 296 families of cosets. We cannot repeat that. What we can do is one coset, to see what one costs and whether the pieces are in place. This section reports that one coset, the superflip’s.

8.1 Cosets

The subgroup is generated by ten of the eighteen moves: the three turns of the top face, the three of the bottom face, and the half turns of the other four. Call them the ten. They generate Reid’s `H`, met above. A coset of `H` is the set of positions reached by playing the ten from a fixed position. Every position of the cube lies in exactly one coset.

	count
positions of the cube	43 252 003 274 489 856 000
cosets	2 217 093 120

	count
positions in a coset	19 508 428 800
cosets we did	1

We did the superflip's coset. We had the superflip already, from the lower bound, so it is a good enough example of a coset.

8.2 The coset as one map

A position of a coset is named by three numbers: how the eight top and bottom corners sit, how the eight top and bottom edges sit, and how the four middle edges sit. The map holds one bit for each. The arrangements of the corners go in pairs, and a pair shares one word, so the map is 406 425 600 machine words of forty-eight bits, which is 19 508 428 800 bits.

The search starts at the superflip and plays words, setting the bit of every position of the coset it reaches. When every bit is set the theorem follows. Thirty-two positions are not reached. We give each of them a word of twenty moves by hand in `RowWits.v`. We do not trust those words: `RowWitsChk.v` plays each one back and asks for the solved cube.

8.3 The reused and the new

We reuse nearly everything: the search, the map, the tables that step a whole map, the ranking of the three numbers, the replay of the leftover words, the phase one table and the program that generates it, the cube, the permutations and the machine-integer tools.

Four things are new, and none of them is about searching.

- The rank and the sign of a permutation on machine integers. Rocq's library has both, but for permutations that cannot be computed.
- That a position of a coset is its three numbers, in both directions. We had one direction only. Without the other the theorem is about triples of numbers and not about the cube.
- The link between the summary the search carries and the position it stands for. The summary is one machine word, the position forty-eight.
- Checks on the tables, one per file, so that each reports separately.

8.4 The unsound stop

Cutting a branch on the table is sound, because a table that says too little only cuts less. Stopping on it is not. Our search used to stop when the table said zero and take the position it had reached as one of the coset. A table of zeros still says too little, so it is still allowed, and it would stop the search everywhere and the theorem would say nothing.

The search now stops on the position. At the bottom it tests the position it is carrying. No part of the proof reads the table, and it costs one comparison.

8.5 Folding the map

The map is 40 320 pages of 20 160 groups, one page for each way the eight top and bottom corners can sit. Sixteen renamings of the cube keep the top and bottom faces in place, send the ten to the ten and leave the superflip alone. Two pages related by a renaming hold the same answer, so one page of each family is enough: 2 768 of the 40 320, a factor of 14.6. A level of the search is one pass over the map, so there is 14.6 times less of it to walk. The price is undoing a renaming whenever a kept page is read.

The kept pages go in pairs of their own, and a pair shares one word as it does on the unfolded side: 1 496 words in place of 2 768. Two hundred and twenty-four of the pages are their own partner and use half a word.

The fold has to be proved as well as written. A renaming sends a member of the coset to a member of the coset. Undoing it gives back the position the page stood for. A map sound after one level is sound after the next. That is the largest single part of the coset's proof.

8.6 The files

the coset and its members	
Row.v	a coset, its members, and each member as a bit
RowMemb.v, RowMembI.v	the cube a member names, and the member a cube gives
RowMembChk.v	that bridge, with nothing left open
RowLeaf.v, RowInH.v	a position of H is its three ranks
RowInst.v, RowReal.v	the instance: the superflip's own coset
ranking, and the moves	
Lehmer.v	the rank and the sign of a permutation, on machine integers
RowUp8ok.v RowUp4inv.v	.. unranking is a permutation, and undoes the ranking
RowPar8.v, RowPar4.v, RowParity.v	the parity tables, and how a move shifts a parity
RowPartC.v, RowPartM.v, RowPartU.v	each of the three parts is a permutation
RowMoveH.v, RowMoveC.v, RowMoveM.v, RowMoveU.v	a move of H is its three halves, each following its table
RowTab.v, RowTabL.v, RowTabP.v, RowTabF.v	the tables themselves, and the checks they pass
the map and the search	
RowMap.v	the map of a coset, and the pass that steps it
RowRun.v	the search, the level loop, and what each owes
RowLvl.v	the pass again, one chunk a page instead of one a word
RowMask.v	the folded phase one table, and the moves worth trying
RowSrch.v, RowSrchP.v	the search with the cuts and the early stop, and its proof
RowMark.v	the leftover words marked into the map the run leaves
RowWits.v, RowWitsChk.v	those words, and the replay that checks them
RowFinal.v	every member of the coset is within twenty moves
the fold	
RowFold.v	the map folded by the sixteen renamings

the fold	
RowFoldSym.v, RowFoldConj.v	the fold tables are the renamings, and they conjugate
RowFoldPart.v, RowFoldSrc.v, RowFoldGath.v	a page renamed, then moved
RowFoldWrite.v, RowFoldLvl.v	what one write costs, and that a level keeps the map sound
RowFoldMem.v, RowFoldOk.v	two members that fold together stand or fall together
RowFoldTot.v, RowFoldPorb.v	the fold tables land in range at every index
RowFoldEmpty.v, RowFoldFinal.v	the map the run starts from, and the one it leaves
RowFoldRun.v, RowFoldSrch.v	the folded search and the folded run are sound
RowFoldSrchI.v, RowFoldSrchIP.v	the same search with the depth as an int, and the two are equal
the two runs	
RowCub.v, RowCubi.v, RowCubInst.v	a position as twenty cubies, carried through the search
RowCubDef.v, RowFoldCubDef.v, RowFoldCubDefI.v	what each run needs, and not one proof
RowCubBoolI.v, RowFoldCubBoolI.v	the two runs: one boolean each, and nothing else
RowCubReal.v, RowFoldCubReal.v	the coset on each map, with only that boolean left open
RowCubProof.v, RowCubProofI.v, RowFoldCubProof.v, RowFoldCubProofI.v	what a true boolean buys
RowCubDoneI.v, RowFoldCubDoneI.v	run and proof joined: the two theorems

8.7 The cost

The coset adds sixty-nine hand-written Rocq files and 17 386 lines to the work above, besides the generated tables.

The search ran twice, over the folded map and over the unfolded one, with the same search in both, and both times it filled the map.

the run	wall clock	processor time	peak memory
over the folded map	5 h 53	5 h 51	–
over the unfolded map	7 h 52	7 h 48	23.3 GB

The fold is worth 1.3 times on the wall clock and 1.3 times on processor time. The run does not follow the size of the map, which is 13 times smaller. Most of the work is the search at the deepest levels, and that is the same tree on both sides. What the map sets is the memory. The unfolded map is 3.25 GB against 248 MB, and a level reads one map while it writes the other, so the unfolded run needed

23.3 GB of a 62 GB machine. Told to collect harder, the same run peaks at 13.9 GB and takes four per cent longer.

Two earlier folded runs say where the time went. Over words of twenty-four bits, holding one corner arrangement each, the run took 6 h 00. The same run with the depth left as a unary numeral instead of a machine integer took 8 h 13. Counting in unary is the enemy here too.

The phase one table is 2.9 GB of Rocq source and 4.5 GB once checked, 8 h 48 of processor time. It is generated once and shared with the lower-bound work.

The statement has no hypothesis left.

```
Theorem real_row_superflip_fold_runi m :
  m \in H -> superflip * m \in ball Sset 20.
```

H is the group of the ten, and a coset is one of its cosets. Every position of the superflip's coset is within twenty moves. Rocq reports only the primitives of its machine-integer and array interface. The unfolded run proves the same statement from its own map.

9 Conclusion

We prove two lower bounds. No position of the cube is solved in 19 face turns, and none in 25 quarter turns. In each case one position is the witness, the superflip for the first and the four-spot with the superflip on it for the second. In each case the proof is a search that comes back empty over a table of estimated distances.

We do not prove that 20 and 26 always suffice. The upper half is a different computation. It is not one search that finds nothing, but two billion searches that must each find everything. We did one of the two billion, and the section above says what it cost.

The three share a trunk. The cube itself, as permutations of the forty-eight facelets, with the eighteen moves and the group they generate. Balls, and what it means for a position to be within d moves. The abstract search, whose contract is that a false answer is a proof. The rule that any summary of a position, together with any table that passes one check, gives an estimate that is never too big. And the tables themselves, held as machine integers in arrays rather than as lists of unary numbers.

What each of the three needed of its own:

- **The twenty face turns.** The phase one summary, which is the edge flips and the slice, its table, and the certificate that checks the table. Three viewing angles of the same search, and seventeen pieces run side by side.
- **The twenty-six quarter turns.** Reid's Proposition 2, which is the one piece of the development argued by hand rather than computed, and the parity argument that turns 25 into 24. A second and much larger summary, 29 billion values, with a table of its own and a sweep of its own.
- **One coset.** A coset held as a map of bits rather than as a tree of positions. That needs the rank and the sign of a permutation on machine integers, the bijection between a position and its three ranks, a pass that steps a whole map one move at a time, the fold by the sixteen renamings, and words supplied by hand for the positions the search does not reach.

The prover is not far behind a program. The same search written in OCaml is about three times faster. We ran both at radius 19 on the reference machine. The OCaml program visits 146 065 078 152 positions in 26.4 processor-hours, which is 0.65 microseconds a position. Rocq takes 87.6 processor-hours over the same tree, which is 2.16. A factor of **3.3**.

We do not assume that the two walk the same tree. Dividing each of the seventeen Rocq pieces by the positions its OCaml counterpart visited gives between 1.98 and 2.52 microseconds, over pieces that differ in size by a factor of two. A Rocq search that cut differently anywhere would show as

scatter there, and there is none. So the run takes a night because the tree holds 146 billion nodes, not because the prover is slow: in OCaml the same tree still costs 26 processor-hours.

The whole development, counted in hand-written Rocq and leaving out the generated tables, is 37 898 lines. Each line of the table counts what that piece adds to the ones above it.

	files	lines
the superflip, for the twenty face turns	53	14 504
the four-spot, for the twenty-six quarter turns	18	6 008
one coset of the upper bound	69	17 386
in all	140	37 898

This development was written with the help of Claude, Anthropic’s coding assistant.

The sources are at github.com/thery/DoubleCover/code/Rubik, with the note, its figures and Reid’s transcribed post beside them.

References

1. Rokicki, T., Kociemba, H., Davidson, M., Dethridge, J.: The Diameter of the Rubik's Cube Group Is Twenty. *SIAM Journal on Discrete Mathematics*. 27, 1082–1105 (2013). <https://doi.org/10.1137/120867366>
2. The Rocq Development Team: The Rocq Prover, <https://rocq-prover.org/>
3. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo, 2021, <https://doi.org/10.5281/zenodo.4457887>
4. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. *Artificial Intelligence*. 27, 97–109 (1985). [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)
5. Culberson, J.C., Schaeffer, J.: Pattern Databases. *Computational Intelligence*. 14, 318–334 (1998). <https://doi.org/10.1111/0824-7935.00065>
6. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. pp. 700–705 (1997)
7. Kociemba, H.: The Two-Phase Algorithm, <https://kociemba.org/cube.htm>
8. Armand, M., Grégoire, B., Spiwack, A., Théry, L.: Extending Coq with Imperative Features and its Application to SAT Verification. In: *Interactive Theorem Proving (ITP 2010)*. pp. 83–98. Springer (2010)
9. Reid, M.: superflip composed with four spot. Post to the Cube-Lovers mailing list, 2 August 1998, The address cited by Rokicki et al. no longer resolves; this is a transcription, <https://github.com/thery/DoubleCover/blob/main/doc/reid-1998-fourspot.md>